

```
/*  
    Assignment no:-4  
*/
```

```
#include<iostream>  
#include<string.h>  
using namespace std;
```

```
class Tree  
{  
    struct node  
    { char data;  
      node *left,*right;  
    };  
    node *root;  
public:  
    Tree()  
    {  
        root=NULL;  
    }  
    void convert_post();  
    void convert_pre();  
    void pre(node *);  
    void in(node *);  
    void post(node *);  
    void preN();  
    void inN();  
    void postN();  
    node *return_root()  
    {  
        return root;  
    }  
};
```

```
void Tree::convert_post()  
{  
    node *s[100];  
    int top=-1;  
    char post[100],token;  
    cout<<"\nEnter postfix Expression:";  
    cin>>post;  
    for(int i=0;post[i]!='\0';i++)  
    {  
        token=post[i];  
        node *newnode=new node;  
        newnode->data=token;  
        newnode->left=newnode->right=NULL;
```

```
        if(isalnum(token))  
            s[++top]=newnode;  
        else  
        {  
            newnode->right=s[top--];  
            newnode->left=s[top--];  
            s[++top]=newnode;
```

```

    }
    root=s[top];
}
}
void Tree::convert_pre()
{
    node *s[100];
    int top=-1;
    char pre[100],token;
    cout<<"\nEnter prefix Expression:";
    cin>>pre;
    for(int i=strlen(pre)-1;i>=0;i--)
    {
        token=pre[i];
        node *newnode=new node;
        newnode->data=token;
        newnode->left=newnode->right=NULL;
        if(isalnum(token))
            s[++top]=newnode;
        else
        {
            newnode->left=s[top--];
            newnode->right=s[top--];
            s[++top]=newnode;
        }
        root=s[top];
    }
}

```

```

void Tree::pre(node *temp)
{
    if(temp!=NULL)
    { cout<<"\t"<<temp->data;
      pre(temp->left);
      pre(temp->right);
    }
}

```

```

void Tree::in(node *temp)
{
    if(temp!=NULL)
    { in(temp->left);
      cout<<"\t"<<temp->data;
      in(temp->right);
    }
}

```

```

void Tree::post(node *temp)
{
    if(temp!=NULL)
    { post(temp->left);
      post(temp->right);
      cout<<"\t"<<temp->data;
    }
}

```

```

void Tree::preN()
{
    node *s[100],*temp=root;
    int top=-1;
    while(temp!=NULL || top>-1)
    {
        while(temp!=NULL)
        {
            cout<<"\t"<<temp->data;
            s[++top]=temp;
            temp=temp->left;
        }
        temp=s[top--];
        temp=temp->right;
    }
}

```

```

void Tree::inN()
{
    node *s[100],*temp=root;
    int top=-1;
    while(temp!=NULL || top>-1)
    {
        while(temp!=NULL)
        {
            s[++top]=temp;
            temp=temp->left;
        }
        temp=s[top--];
        cout<<"\t"<<temp->data;
        temp=temp->right;
    }
}

```

```

void Tree::postN()
{
    node *s[100],*temp=root;
    char post[100];
    int i=0,top=-1;
    while(temp!=NULL || top>-1)
    {
        while(temp!=NULL)
        {
            post[i++]=temp->data;
            s[++top]=temp;
            temp=temp->right;
        }
        temp=s[top--];
        temp=temp->left;
    }
    for(i=strlen(post)-1;i>=0;i--)
        cout<<"\t"<<post[i];
}

```

```

int main()

```

```

{
    int ch,choice;
    Tree tr;

do
{ cout<<"\n1:Create Expression tree using postfix.";
  cout<<"\n2:Create expression tree using prefix.";
  cout<<"\n3:Exit.";
  cout<<"\nEnter your choice:";
  cin>>ch;

    if(ch==1)
        tr.convert_post();
    else if(ch==2)
        tr.convert_pre();
    else
        return 0;

    cout<<"\n1:Use recursive traversals.";
    cout<<"\n2:Use non-recursive traversal.";
    cout<<"\nEnter your choice:";
    cin>>choice;

    switch(choice)
    {
    case 1:
        cout<<"\nRecursive Preorder:";
        tr.pre(tr.return_root());
        cout<<"\nRecursive Inorder:";
        tr.in(tr.return_root());
        cout<<"\nRecursive Postorder:";
        tr.post(tr.return_root());
        cout<<endl;
        break;

        case 2:
            cout<<"\nPreorder:";
            tr.preN();
            cout<<"\nInorder:";
            tr.inN();
            cout<<"\nPostorder:";
            tr.postN();
            cout<<endl;
            break;
        }
    }while(ch<3);
};

/*
outout

```

1:Create Expression tree using postfix.
 2:Create expression tree using prefix.
 3:Exit.
 Enter your choice:1

Enter postfix Expression:ab+c*

1:Use recursive traversals.

2:Use non-recursive traversal.

Enter your choice:1

Recursive Preorder: * + a b c

Recursive Inorder: a + b * c

Recursive Postorder: a b + c *

1:Create Expression tree using postfix.

2:Create expression tree using prefix.

3:Exit.

Enter your choice:2

Enter prefix Expression:

*+abc1:Use recursive traversals.

2:Use non-recursive traversal.

Enter your choice:2

Preorder: * + a b c

Inorder: a + b * c

Postorder: a b + c *

1:Create Expression tree using postfix.

2:Create expression tree using prefix.

3:Exit.

Enter your choice:3

*/